

Relatório 6 - Prática de formulários e dashboard

Kaíque Medeiros Lima

1. Descrição da atividade

a. Formulários com React Hook Form

A atividade continua o projeto de Fórmula 1, agora com mais atenção ao funcionamento dos formulários de login e cadastro. As telas usam React Hook Form para registrar os campos, tratar o envio e acessar os erros. Em vez de concentrar toda a verificação dentro do botão, cada campo é conectado ao formulário pelo register, e o handleSubmit organiza o momento de chamar a função de envio.

O useForm recebe o tipo dos dados e o zodResolver, que faz a ligação com as regras do Zod. No login, os campos são e-mail e senha. No cadastro, também entram o nome e a confirmação de senha. Os formulários usam noValidate para deixar essa verificação por conta da integração definida no código, mantendo as mensagens no mesmo padrão visual da aplicação.

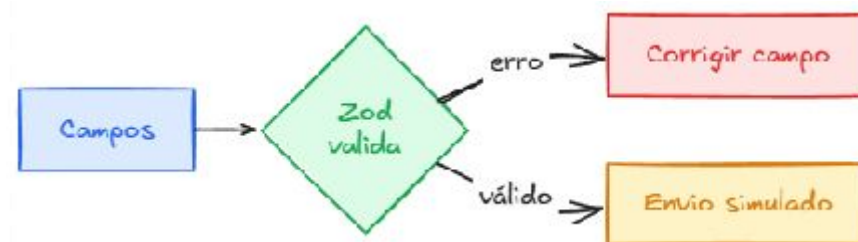


Figura 1 - Fluxo de validação e envio simulado. Autoria própria, Excalidraw.

b. Schemas e validação dos dados

As regras ficam no arquivo schemas.ts, separado das telas. O loginSchema exige um e-mail válido e uma senha com pelo menos seis caracteres. O registerSchema acrescenta um nome com no mínimo três caracteres e a confirmação da senha. Depois, o refine compara as duas senhas e associa o erro ao campo confirmPassword quando os valores são diferentes.

Essa separação ajuda a entender o que cada formulário aceita sem ter que procurar as regras no meio do JSX. Os tipos ILoginSchema e IRegisterSchema são obtidos com z.infer a partir dos próprios schemas. Assim, o formato usado pelo TypeScript acompanha a definição da validação. Ainda assim, a regra de três caracteres para o nome verifica o tamanho do texto, não se ele representa realmente um nome completo.

```
9
10 export const registerSchema = z
11   .object({
12     name: z.string().min(3, "Informe seu nome completo"),
13     email: z.string().email("Informe um e-mail válido"),
14     password: z.string().min(6, "A senha deve ter no mínimo 6 caracteres"),
15     confirmPassword: z.string().min(1, "Confirme sua senha"),
16   })
17   .refine((data) => data.password === data.confirmPassword, {
18     message: "As senhas precisam ser iguais",
19     path: ["confirmPassword"],
20   });
21
22 export type IRegisterSchema = z.infer<typeof registerSchema>;
```

Figura 2 - Schema de cadastro e comparação das senhas. Código da branch do Card 06.

c. Mensagens, carregamento e componentes compartilhados

O componente `TextField` foi preparado para receber uma mensagem de erro opcional. Quando existe um problema, a mensagem aparece em vermelho abaixo do campo correspondente. O label continua ligado ao input pelo id, e as telas utilizam `autoComplete` para indicar o tipo de preenchimento esperado. O formulário consegue, então, mostrar com mais clareza qual informação precisa ser corrigida.

No envio válido, o estado `isLoading` passa para `true`, a mensagem anterior é limpa e o botão fica desabilitado. O texto muda para `Entrando` ou `Criando conta` enquanto um `setTimeout` simula a espera de um segundo. Ao terminar, aparece uma confirmação com `role="status"`. Isso representa o fluxo visual de uma operação assíncrona, mas não existe uma requisição de login ou cadastro para um servidor nessa etapa.

d. Dashboard da temporada de Fórmula 1

A rota `/dashboard` apresenta uma visão resumida da temporada de 2021. O conteúdo é dividido em cards de indicadores, classificação de pilotos, destaque do campeão e últimas corridas. Os dados estão escritos no próprio componente e servem como exemplos para a montagem da interface. Dessa forma, a tela pode ser construída e organizada antes de depender de uma API.

O layout utiliza grid e classes responsivas do Tailwind. Os indicadores passam a ocupar três colunas em telas médias, e os blocos de classificação e campeão ficam lado a lado em telas maiores. Essa organização deixa os números principais mais fáceis de localizar e reserva outras áreas para informações com mais texto. O dashboard dessa branch apresenta dados estáticos, sem uma ação de atualização conectada a um serviço externo.

e. Organização do projeto e alcance da prática

A estrutura mantém as rotas em app, as funcionalidades em modules e os componentes reaproveitados em shared. LoginScreen e RegisterScreen cuidam dos formulários, DashboardScreen reúne o painel e os schemas ficam no módulo de autenticação. Essa divisão permite evoluir uma parte sem colocar todas as responsabilidades dentro de uma única página.

O README também deixa claro que as ações são simulações locais. O link para o dashboard de exemplo está disponível na tela de login, e o projeto não implementa autenticação real ou armazenamento de usuários. A prática está voltada para validação, retorno visual e organização da interface, que são partes importantes da experiência antes da integração com um backend.

2. Conclusões

O card mostra como um formulário precisa cuidar tanto dos dados quanto da resposta que oferece ao usuário. React Hook Form organiza os campos e o envio, enquanto o Zod concentra as regras e permite aproveitar os mesmos schemas na definição dos tipos. As mensagens e o estado de carregamento deixam o fluxo mais claro. No dashboard, a divisão dos indicadores e o uso de componentes mantêm a interface organizada, preparando a base para trabalhar com dados externos nos próximos passos.

3. Referências

<https://react-hook-form.com/>

<https://zod.dev/>

React Hook Form - Complete Tutorial (with Zod)

https://www.youtube.com/watch?v=er_QPBldsXE

<https://www.youtube.com/watch?v=PqRe0Q9b5QI>